# Object Oriented programming

- OOPS stand for object oriented programming.
- An object-oriented programming language uses objects in its programming.
- Programming with object-oriented concepts aims to emulate real-world concept such as inheritance, polymorphism, abstraction etc. in a program:

## • Classes & objects

### Classes

"Classes are user-defined data-Type and are a template for creating object."

- class consist of variable and functions which are also called class member.
- class is like a blueprint of these entities

Syntax: class class_Name
{
// body of the class
};

keyword

// body of the class → properties
// body of the class → Methods

### o Objects

" Object are instances of a class. To create an object, We just have do specify the class Name and then the object Name."

- Object can Access class attributes and method which are bound to the class definition.
- Object are real entities in the real world.

**Syntax:**

```
class  class-Name
{
    //  body of the class
};
int main()
{
    class-Name  object Name ;  // object
}
```

## • class Attributes & Methods

class Attributes & method are variable and function that are defined inside the class. They are also known as class members.

**example:**

```
# include<iostream>
using namespace, std ;
class Employee
{
    int ID ;
    char name ;
    public :
};
int main () {
    Employee Sagar ;
Sagar. ID = " 205 " ;
Sagar. Name = " Rathod " ;
 Cout << Sagar. Name ;
} return 0 ;
```

**Output:**

Rathod

## Encapsulation:

"Encapsulation is wrapping up of data & member function in a single unit called class."

- Encapsulation is the first pillar of object-oriented program
- it means wrapping up data attributes and method toghether.
- The goal is to keep sensitive data hidden from user.

example:

```
#include <iostream.h>
using namespace std;

class Encapsulation
{
    private :          <-- Access modifire
        int s;
public :
    void setS (int a){
        s = a;
    }
    void getS {
        cout return s;
    }
};
int main () {
    Encapsulation obj;      <-- object creation
    obj.setS (10);          <-- using object function call
    obj.getdata
    cout << obj.getS ();
}
```

Hide data

output:

```
10
```

## Inside function definination

Example:

```
class Employee
{
  public :
     int ID ;
    void getdata() {
          cout << ID << endl ;
    }
};
```

← inside

## outsicle function definination

example:

```
class Employee
{
  public :
     int ID
        void getdata() ;
};

void Employee::getdata()
{
     cout << ID << endl ;
}
```

← outside

## • Access Modifiers

private : data & methods accessible inside class

public : data & memod accesible to everyone

protected : data & method accessible inside class & to its derived class.

by default all data and methods are private

# Constructor

"Special method invoked automatically at time of object creation."

- Constructors are used to initialize the objects of their class.
- Same name as class
- Constructor doesn't have a return type
- Only called once (automatically), at object creation
- Memory allocation happens when constructor is called.

## Types of constructor

1] Non parameterized -> Default ~~cons~~ constructor

2] parameterize

3] copy ~~contr~~ constructor

- parameterized constructor are those constructors that take one or more parameters.
- Default constructor are those contructors that take no parameters.

## Constructor Overloading

constructor overloading is a concept similar to function overloading At the time of definition of an instance, the constructor, whi will match the number and type of argument, will get executed.

this is a special pointer in c++ that points to the current object

this -> prop is same as *(this).prop

## Copy Constructor

A copy constructor is a type of constructor that create a copy of another object. if no copy constructor is written in a program compiler will supply its own copy constructor.

Syntax:

```
class class_Name
{
    int s;
public:
    class_Name (class_Name &obj)
    {
        a = obj.a;
    }
};
```

## Destructor

-> opposite of constructor

deallocate — memory allocate

Syntax:

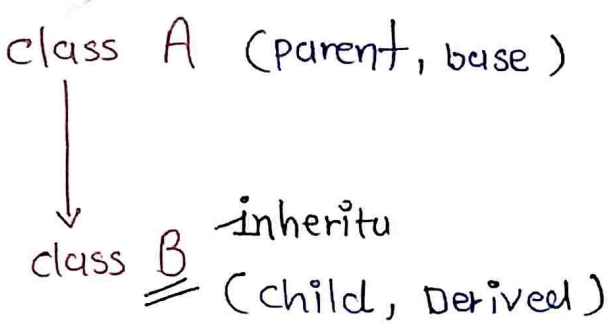
```
~ class_Name ( ) { }
```

- A destructor is also a special member function like a constructor.
- Destroys the class object created by the constructor.
- Destructor release memory space occupied by the object created by the constructor.
- Destructor is called when the function ends.
- example:

```
~ test ( ) {
    cout << " \n Destructor executed " ;
}
```

# Inheritance

When properties & member functions of base class are passed on to the derived class.




—> it use for code reusability.

- Inheritance is one of the most important features of object oriented programming in C++.

## Syntax of Inharitance :

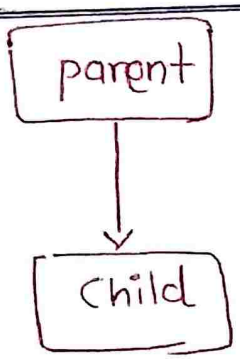
```
class derived_class_name : access-specifier base_class_Name
{
        // body.....

};
```

Example :

```
class ABC : private xyz {....}
class pqr : public svq {....}
class AVVP : protected AWM {....}
```

# Type's of Inheritance

## 1] Single inheritance



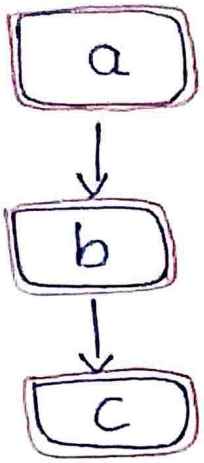

Example :

```
#include <iostream>
// Base class
class Animal {
public :
    void eat()
        std :: cout <<" Eating... " << std :: endl;
    }
};
// Derived class
class Dog :public Animal {
public
    void bark() {
        std :: cout <<" Barking... " << std :: endl;
    }
};
int main() {
    Dog myDog ;
    MyDog.eat(); // inheritance from Animal class
    MyDog.bark(); // specific to Dog class

    return 0;
}
```

output :

```
Eating...
Barking...
```

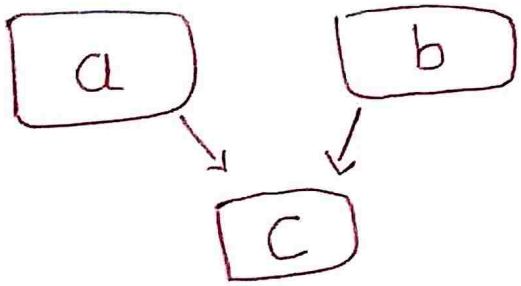
## 2] Multi-Level Inheritence




- Multilevel inheritance is a type of Inheritence in which one derived class is ~~in~~ inharited from another derived class.

```
class    class A
{
        // body of classA
};
 class classB : public classA
{
    // body of classB
};
class  classC : public classB
{
     // body of class C
};
```

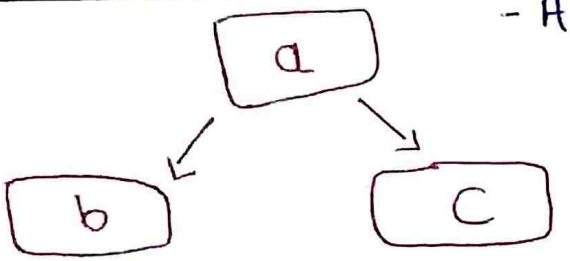
## 3] Multiple Inheritance




- Multiple inheritence is a type of inheritance in which one derived class is inheriteed from more than one base class.

```
class classA
{
  // body classA
};
class classB
{
   body classB
};
class classC : public classA , public classB
{
    // body of class C
};
```

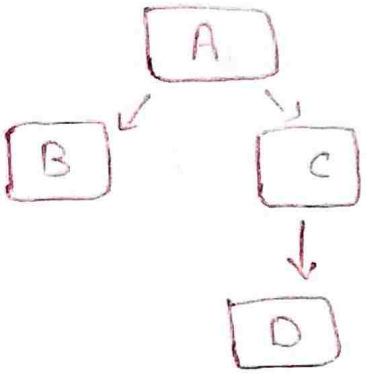
## 4] Hifarchical Inheritance




- A ~~hire~~hierarchical Inheritance is a type of Inheritance in with which several derived classes are Inher.. from a single buse class.

```
Class classA
{
    // body A
};
class   B : public ~~class~~ classA {
      // body B
};
class   C : public ~~B~~A
{
    //  body C
};
```

## 5] Hybrid Inheritance




- Hybrid inheritance is a combination of diffrent types of inheritances.

```
class A
{
    // body A
};
class B
{
    // body B
};
class C : public A, public B
{
    // body C
};
class D : public C
{
    // body of C
};
```

## • Polymorphism

- Poly means Saveral and morphism means form.
- polymorphism is something that has several forms or we can say it as one name and multiple forms.
- There are two types of polymorphism
  - Comple Time polymorphism
  - Run Time ~~pho~~ polymorphism

### 1] compile Time polymorphism

There two Type's of compile - Time polymorphism

1] Function overlodding

2] operator overloading

### 2] run - Time polymorphism

- Function overriding
  - parent & child both contain the same function with diffrent implementation.
  - The parent class function is said to be overridin.

### • Virtual functions

A virtual function is a member function that you expect to be redefineed in derived classes.

- Virtual function are Dynamic in nature.
- Deffned by the keyword " virtual " inside a base class and are always declareed with a base class and overridden in child cass.

## ○ Abstraction

" Hide all unnecessary details & showing only the importance parts."

— Abstraction base Access Modifires

using Abstract classes

- Abstract classes are used to provide a base class from which other classes can be derived.
- They connot be instantiated cnet are meut to be inhe... obj.
- Abstract classes are typically used to define an interface for derived classes.

## ○ Friend function & classes

— ~~Freind~~ Friend function is function that is declared as a friend of a class not as a member of a class instead of that it can access private & protected member of, class.

Syntax:

```
class class-name
{
    friend return-type function-name (argument);
};

return-type class-name :: function-name (argument);
{
        // body of the function
}
```

# Friend classes

Friend class is a class that granted accessibility of private and protected.

Syntax:

```
class   class_name
{
   private :

   public :
      friend class   class-name;
};
```